Big O

- Calculate no of iterations
- Neglecting lower terms
- " co-efficients.

Qu # itr = $2n + 100$ ⟶ $O(n)$ Ans

Qu $5n^2 + 3n + 500$ ⟶ $O(n^2 + n)$ X

$O(n^2)$ ✓ Ans

```
fun(n) {
    res = 0
    for(i=1 ———— n)
        for(j=1 ———— n) {
            }
    }
    for(i=1 ———— n) {
    }
    for(i=1 ———— 2^n) {
    }
}
```

n^2 itr

n itr

2^n itr

itr = $n^2 + n + 2^n = O(2^n)$

```
for(i=1 ———— n)
    for(j=1 ———— n) {
        }
    }
```

i	j [1-n]	itr
1	[1, n]	n.
2	[1, n]	n
3	[1, n]	n.
⋮	⋮	⋮
n	[1, n]	n

n times

$n * n = n^2$

Qu

```
for (i=1 ———————— n) {
        .......
}
for (i=1 ———————— m) {
        .........
}
```

n

m

itr = n+m

= O(n+m)

Qu

```
fun(n) {
    for (i = 1 ———————— 100) {
        ———————————
    }
}
```

100

O(1) Ans

Qu

```
fun(n) {
    i = 0
    while (i < n) {
        i = i *2;
    }
}
```

n = 100

i=0 , i = 0*2 = 0

i=0 , i = 0*2 = 0

O(∞) —— infinite loop

i = 0

Qu

```
for(i=0 ———————— n-1) {
    for(j=0 ———————— i) {

    }
}
```

Bharath – O(n) ✗

i	j [0, i]	# itr
0	[0, 0]	1
1	[0, 1]	2
2	[0, 2]	3
⋮	⋮	⋮
n-1	[0, n-1]	n.

$1 + 2 + 3 \cdots\cdots n$

$\frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}.$

$= O(n^2)$

$\# \text{itr} = \frac{n^2}{2} + \frac{n}{2}$

Qu

```
for(i=1; i<=100; i=i*2) {
    for(j=1; j<=n; j++) {
        ..............
    }
}
```

i	j [1, n]	itr
1	[1, n]	n.
2	[1, n]	n
4	[1, n]	n
8	[1, n]	n
16	[1, n]	n
32	[1, n]	n
64	[1, n]	n

$7n = O(n)$

Qu

```
for(i=1; i<=n; i=i*2) {
    for(j=1; j<=n; j++) {
        ..............
    }
}
```

	i	j [1, n]	itr
2^0	1	[1, n]	n.
2^1	2	[1, n]	n
2^2	4	[1, n]	n
2^3	8	[1, n]	n
2^4	16	[1, n]	n
2^5	32	[1, n]	n
	64	[1, n]	n
	⋮	⋮	⋮
2^K	n.	[1, n]	n.

?

$$n * [\log_2 n]$$

$$2^K \simeq n.$$

Apply $\log_2$ both sides

$$\log_2 2^K = \log_2 n$$

$$\boxed{K = \log_2 n}$$

Qu

$\log n$

n

$n \log n$

$n!$ ✓

Qu

$n!$ $\quad 2^n \quad n^n \quad \log n$

$$\log n. < 2^n < n! < n^{n.}$$

Qu

```
f1(n) {
    for(i=1 ———— n) {
        j=1
        while(j<n) {
            j=j*2;
        }
    }
```

$n * \log_2 n.$

```
f2(n)
    for(i=1 ———— n) {

    }
```

$O(n)$

```
f3(n)
    for(i=1 ———— n) {
        for(j=1 ———— n) {
    }   }
```

$O(n^2)$

```
f4(n)
    i=1
    while(i<=n) {
        i=i*2
    }
}
```

$O(\log_2 n)$

$n \qquad n^2 \qquad \log_2 n \qquad n\log_2 n$

$\log_2 n < n < n\log_2 n < n^2.$

Qu

```
x = 10
for(i=1 ———————— 5) {
    x -=2;
    if(x%3==0) {
        continue;
    }
    print(x);
}
```

8 4 2

i	x [10]	
1	x=8	
2	x=6	skipped
3	x=4	
4	x=2	
5	x=0	skipped

Break: 10:26 — 10:35

Qu

```
for(i=1; i!=10; i+=2) {
        print(i)
}
```

i=1 ——— 1
i=3 ——— 3
i=5 ——— 5
i=7 ——— 7
i=9 ——— 9
i=11 ——— 11
⋮ ⋮

Qu

```
for(i=1 ———————————— 4) {
        if(i%2 ==0  &&  i*2<7) {
            print(i);
        }
}
```

i=1 ———
i=2 ——— 2
i=3 ———
i=4 ———

Qu

```
i = 1
while(i < 5) {
        print(i);
        i++ ———— get rid of infinite loop
}
```

Infinite loop

Qu

```
int sum; ———— local var is not initialized
for(i=1 ———————————— 5) {
        sum+=i;
}
print(sum);
```

Qu

```
for(i=1 ———————————— 3) {
    for(j=3 ; j>=1 ; j--) {
        print(i        j)
    }
    println()
    print("\n");
}
```

1	3	1	2	1	1
2	3	2	2	2	1
3	3	3	2	3	1

i=1 ————————

j=3 ———————— 1 3

j=2 ———————— 1 2

j=1 ———————— 1 1

j=0 ———— stop

i=2 ————————

j=3 ———————— 2 3

j=2 ———————— 2 2

j=1 ———————— 2 1

j=0 ———— stop

i=3

⋮

Qu

```
arr = {1.2.3.4}
for(i=0; i<=arr.length; i++) {
    print(arr[i]);
}
```

Qu

```
for(i=2 ; i<=10; i+=2) {
    print(i);
}
```

i=2 ———————— 2

i=4 ———————— 4

i=6 ———————— 6

i=8 ———————— 8

i=10 ———————— 10

i=12

Qu

```
for(i=10; i>0; ~~i++~~ i--) {
        print(i)
}
```